

LAPORAN TUGAS BESAR
MATA KULIAH PENGANTAR DEEP LEARNING
(Sistem Deteksi Status Komponen Mobil)



Disusun Oleh:

Nama	NIM	Kelas
Rio Ramadhani Harllambang	1202220205	SI-46-EDM

Tanggal Pengumpulan: Rabu, 10 Juni 2026

Link Github: https://github.com/Tayo1103/tugas_besar_deep_learning.git

PROGRAM STUDI SISTEM INFORMASI
FAKULTAS REKAYASA INDUSTRI
UNIVERSITAS TELKOM
BANDUNG
2026

DAFTAR ISI

DAFTAR ISI.....	i
DAFTAR GAMBAR.....	iii
DAFTAR TABEL	iv
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	1
1.3 Tujuan	2
1.4 Batasan Masalah.....	2
BAB II PEMBAHASAN.....	3
2.1 Gambaran Umum Sistem	3
2.1.1 Alur Sistem Secara Keseluruhan	3
2.1.2 Spesifikasi Lingkungan Pengembangan.....	4
2.1.3 Alur Penggunaan Aplikasi Deployment.....	5
2.2 Pengumpulan Dataset.....	5
2.2.1 Sumber Data (Simulator 3D)	6
2.2.2 Otomatisasi dengan Selenium	6
2.2.3 Variasi yang Dikumpulkan (sudut, zoom, konfigurasi).....	7
2.2.4 Auto Crop & Preprocessing Screenshot.....	8
2.2.5 Hasil Pengumpulan Dataset	9
2.3 Preprocessing Dataset	10
2.3.1 Validasi Data.....	10
2.3.2 Distribusi Label	11
2.3.3 Verifikasi Ukuran dan Mode Gambar	11
2.3.4 Pembagian Dataset.....	12
2.4 Augmentasi Data	13
2.4.1 Pemilihan Teknik Augmentasi.....	13
2.4.2 Pemilihan Teknik Augmentasi.....	14
2.5 Arsitektur Model	15
2.5.1 Backbone EfficientNet-B0	16
2.5.2 Custom Classification Head	16
2.5.3 Fungsi Loss, Optimizer, dan Scheduler	17
2.6 Training Model	17
2.7 Evaluasi Model.....	19
2.7.1 Metrik Keseluruhan (Macro F1, Micro F1, Precision, Recall, Hamming Loss).....	19

2.7.2	Performa per Label (Classification Report)	20
2.7.3	Confusion Matrix per Label.....	20
2.7.4	Prediksi Sample	21
2.8	Deployment	21
BAB III PENUTUP		25
3.1	Kesimpulan	25
3.2	Saran.....	25
LAMPIRAN		26

DAFTAR GAMBAR

Gambar 2.1 Flowchart Sistem End-to-End Car Part State Detection	3
Gambar 2.2 Flowchart Penggunaan Aplikasi oleh Pengguna.....	5
Gambar 2.3 Tampilan Running Skrip Pengumpulan Dataset	6
Gambar 2.4 Tampilan Folder Dataset Hasil Pengumpulan (2.304 Gambar)	9
Gambar 2.5 Hasil Validasi Dataset	10
Gambar 2.6 Hasil Analisis Distribusi Label Dataset	11
Gambar 2.7 Hasil Verifikasi Ukuran dan Mode Gambar	11
Gambar 2.8 Hasil Split Dataset dan Penyalinan ke Folder Subset	12
Gambar 2.9 Distribusi Label per Subset	13
Gambar 2.10 Implementasi Augmentasi Data Latih.....	15
Gambar 2.11 Implementasi Arsitektur Model CarPartClassifier	16
Gambar 2.12 Implementasi Loss, Optimizer, dan Scheduler.....	17
Gambar 2.13 Hasil Running Proses Pelatihan Model.....	18
Gambar 2.14 Hasil Evaluasi Keseluruhan pada Test Set.....	19
Gambar 2.15 Hasil Classification Report Per-class	20
Gambar 2.16 Hasil Confusion Matrix per Label.....	20
Gambar 2.17 Hasil Visualisasi Prediksi Sampel (Hijau = Benar, Merah = Salah)	21
Gambar 2.18 Halaman Utama Aplikasi Deployment	22
Gambar 2.19 Hasil Prediksi pada Aplikasi Deployment	23
Gambar 2.20 Halaman Hasil Uji Robustness Model	23
Gambar 2.21 Visualisasi Hasil Evaluasi Model.....	24

DAFTAR TABEL

Tabel 2.1 Spesifikasi Lingkungan Pengembangan	4
Tabel 2.2 Variasi Konfigurasi Kondisi pada Pengumpulan Dataset.....	7
Tabel 2.3 Variasi Level Zoom pada Pengumpulan Dataset.....	8
Tabel 2.4 Evaluasi Teknik Augmentasi	14
Tabel 2.5 Konfigurasi Pelatihan Model	18

BAB I PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi kecerdasan buatan, khususnya pada bidang *deep learning* dan *computer vision*, telah membuka peluang baru dalam otomatisasi pengenalan objek secara visual. Salah satu penerapannya adalah kemampuan model untuk memahami kondisi atau state dari suatu objek hanya berdasarkan citra visual, tanpa memerlukan sensor fisik maupun intervensi manusia secara langsung.

Dalam konteks industri otomotif, pemantauan kondisi kendaraan secara visual memiliki nilai praktis yang tinggi. Informasi seperti apakah pintu kendaraan dalam keadaan terbuka atau tertutup, maupun kondisi kap mesin, merupakan informasi penting yang dapat dimanfaatkan dalam berbagai skenario, mulai dari sistem keamanan kendaraan, inspeksi otomatis di lini produksi, hingga aplikasi *smart parking* dan *fleet management*.

Pendekatan konvensional untuk mendeteksi kondisi seperti ini umumnya mengandalkan sensor mekanik atau elektrik yang terpasang langsung pada kendaraan. Pendekatan tersebut memiliki keterbatasan dari sisi biaya pemasangan, kebutuhan kalibrasi berkala, serta ketergantungan pada kondisi perangkat keras. Sebaliknya, pendekatan berbasis *computer vision* hanya memerlukan kamera sebagai perangkat input, menjadikannya lebih fleksibel dan berpotensi lebih hemat biaya.

Penelitian ini mengusulkan pembangunan model *deep learning* berbasis *multi-label image classification* yang mampu memprediksi kondisi lima bagian kendaraan sekaligus, yaitu pintu depan kiri, pintu depan kanan, pintu belakang kiri, pintu belakang kanan, dan kap mesin, dari sebuah citra tunggal. Model dikembangkan menggunakan arsitektur EfficientNet-B0 dengan pendekatan *transfer learning*, serta dilatih menggunakan dataset yang dikumpulkan secara otomatis dari simulator mobil 3D interaktif berbasis web.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana membangun pipeline pengumpulan dataset secara otomatis dari simulator mobil 3D interaktif berbasis web?
2. Bagaimana merancang dan melatih model *deep learning* yang mampu memprediksi kondisi lima bagian kendaraan secara bersamaan (*multi-label classification*)?
3. Bagaimana tingkat performa dan robustness model terhadap variasi sudut pandang dan skala tampilan kendaraan?

1.3 Tujuan

Tujuan dari penelitian ini adalah sebagai berikut:

1. Membangun sistem pengumpulan dataset otomatis menggunakan Selenium WebDriver yang mampu menghasilkan dataset citra kendaraan dengan variasi konfigurasi, sudut pandang, dan skala yang beragam.
2. Merancang dan melatih model *multi-label classification* berbasis EfficientNet-B0 yang mampu memprediksi kondisi terbuka atau tertutup dari lima bagian kendaraan secara simultan dalam satu inferensi.
3. Mengevaluasi performa model menggunakan metrik yang relevan untuk *multi-label classification*, meliputi *Macro F1-Score*, *Micro F1-Score*, *Precision*, *Recall*, dan *Hamming Loss*.

1.4 Batasan Masalah

Agar penelitian ini terfokus dan dapat diselesaikan secara sistematis, ditetapkan batasan masalah sebagai berikut:

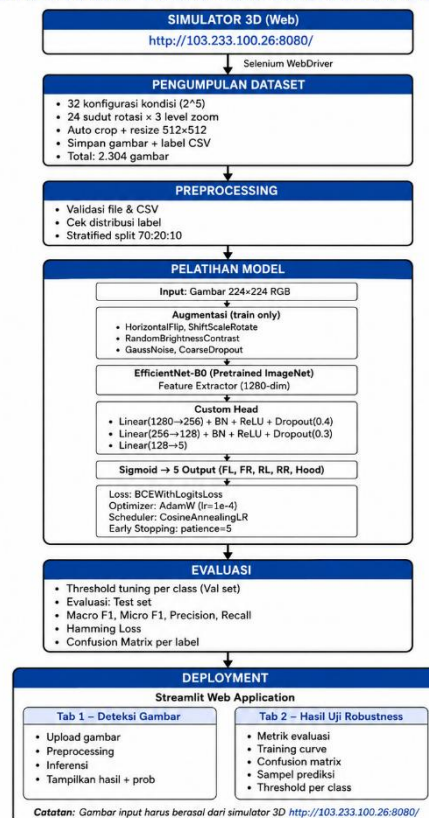
1. Dataset yang digunakan seluruhnya bersumber dari simulator mobil 3D interaktif berbasis web yang dapat diakses pada alamat <http://103.233.100.26:8080/>, sehingga model tidak diujikan pada kendaraan nyata.
2. Model hanya mendeteksi kondisi dari satu model kendaraan, yaitu BMW X7, sesuai dengan model yang tersedia pada simulator.
3. Prediksi dilakukan terhadap lima label kondisi, yaitu pintu depan kiri (*front left door*), pintu depan kanan (*front right door*), pintu belakang kiri (*rear left door*), pintu belakang kanan (*rear right door*), dan kap mesin (*hood*), dengan dua kemungkinan nilai untuk masing-masing label, yaitu terbuka (1) atau tertutup (0).
4. Arsitektur model yang digunakan dibatasi pada EfficientNet-B0 dengan *custom classification head* berbasis *fully connected layer*.
5. Proses pelatihan dilakukan pada lingkungan lokal tanpa akselerasi GPU.

BAB II PEMBAHASAN

2.1 Gambaran Umum Sistem

Sistem yang dibangun dalam penelitian ini merupakan pipeline end-to-end yang terdiri dari lima tahap utama, yaitu pengumpulan dataset, preprocessing, pelatihan model, evaluasi, dan deployment. Gambaran umum alur sistem dapat dilihat pada Gambar 2.1.

Flowchart Sistem Deteksi Multi-Label dari Simulator 3D



Gambar 2.1 Flowchart Sistem End-to-End Car Part State Detection

2.1.1 Alur Sistem Secara Keseluruhan

Secara garis besar, sistem bekerja melalui alur sebagai berikut:

1. Tahap 1 (Pengumpulan Dataset)

Pengumpulan dataset dilakukan secara otomatis menggunakan skrip berbasis Selenium WebDriver yang mengakses simulator mobil 3D interaktif berbasis web. Skrip secara sistematis mengiterasi seluruh kombinasi konfigurasi kondisi kendaraan (32 kombinasi dari 5 bagian dengan nilai biner), lalu untuk setiap konfigurasi melakukan variasi sudut pandang horizontal sebanyak 24 sudut dengan interval 15° dan variasi zoom pada 3 level. Setiap frame yang dihasilkan di-screenshot, di-crop otomatis untuk menghilangkan elemen UI browser, kemudian disimpan beserta label kondisinya dalam file CSV.

2. Tahap 2 (Preprocessing)

Data hasil pengumpulan divalidasi terlebih dahulu untuk memastikan tidak ada file yang hilang, corrupt, atau duplikat. Selanjutnya dataset dibagi menjadi tiga subset menggunakan stratified split berdasarkan kombinasi label, dengan proporsi 70% data latih (*train*), 20% data validasi (*val*), dan 10% data uji (*test*).

3. Tahap 3 (Pelatihan Model)

Model dilatih menggunakan arsitektur EfficientNet-B0 dengan *custom classification head*. Pada tahap ini diterapkan augmentasi data untuk meningkatkan robustness model terhadap variasi visual. Proses pelatihan menggunakan fungsi loss *Binary Cross Entropy with Logits* (BCEWithLogitsLoss), optimizer AdamW, dan scheduler CosineAnnealingLR, disertai mekanisme *early stopping* untuk mencegah overfitting.

4. Tahap 4 (Evaluasi)

Model terbaik yang tersimpan selama proses pelatihan dievaluasi pada data uji menggunakan metrik *Macro F1-Score*, *Micro F1-Score*, *Precision*, *Recall*, dan *Hamming Loss*. Sebelum evaluasi akhir, dilakukan *threshold tuning* per kelas menggunakan data validasi untuk mendapatkan nilai threshold optimal yang memaksimalkan F1-Score tiap label.

5. Tahap 5 (Deployment)

Model terbaik hasil pelatihan selanjutnya di-deploy dalam bentuk aplikasi web interaktif menggunakan framework Streamlit. Aplikasi ini memungkinkan pengguna untuk mengunggah gambar kendaraan secara langsung melalui antarmuka web, kemudian sistem akan memproses gambar tersebut dan menampilkan hasil prediksi kondisi kelima bagian kendaraan secara real-time. Setiap prediksi ditampilkan beserta nilai probabilitasnya untuk masing-masing label. Selain fitur deteksi, aplikasi juga menyediakan halaman khusus yang menampilkan hasil evaluasi model secara lengkap, meliputi kurva pelatihan, confusion matrix per label, serta sampel visualisasi prediksi pada data uji.

2.1.2 Spesifikasi Lingkungan Pengembangan

Sistem dikembangkan dan dijalankan pada lingkungan dengan spesifikasi pada Tabel 2.1 berikut.

Tabel 2.1 Spesifikasi Lingkungan Pengembangan

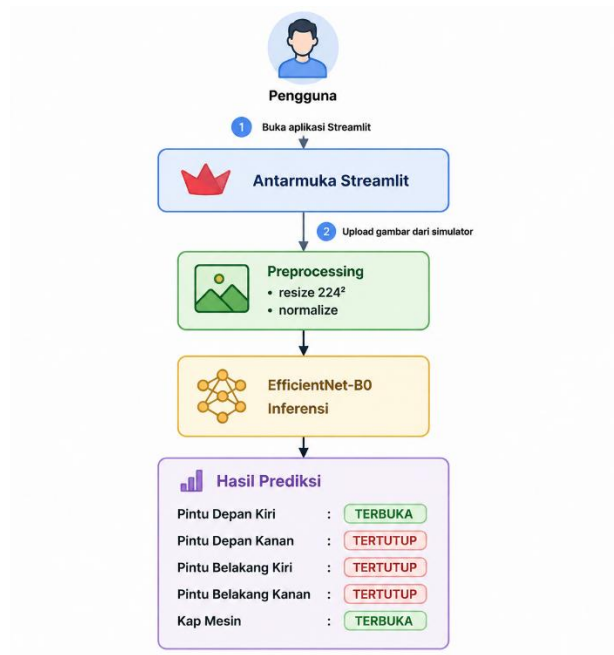
Komponen	Spesifikasi
Sistem Operasi	Windows 11
Prosesor	Intel (CPU only, tanpa GPU)
Bahasa Pemrograman	Python 3
Deep Learning Framework	PyTorch
Arsitektur Model	EfficientNet-B0 via timm
Augmentasi	Albumentations
Otomatisasi Browser	Selenium WebDriver + ChromeDriver
Antarmuka Notebook	VS Code

2.1.3 Alur Penggunaan Aplikasi Deployment

Setelah model selesai dilatih dan dievaluasi, model di-deploy menggunakan Streamlit sebagai antarmuka pengguna. Alur penggunaan aplikasi adalah sebagai berikut:

1. Pengguna mengakses aplikasi Streamlit melalui browser
2. Pengguna membuka tab Deteksi Gambar
3. Pengguna mengambil screenshot dari simulator 3D pada alamat <http://103.233.100.26:8080/> dengan konfigurasi kondisi kendaraan yang diinginkan
4. Pengguna mengunggah gambar tersebut melalui tombol *upload* yang tersedia
5. Sistem secara otomatis melakukan preprocessing gambar (resize ke 224×224, normalisasi ImageNet)
6. Model melakukan inferensi dan menghasilkan probabilitas untuk kelima label
7. Probabilitas dibandingkan dengan threshold optimal hasil tuning per kelas
8. Aplikasi menampilkan hasil prediksi kondisi setiap bagian kendaraan beserta nilai probabilitasnya

Alur tersebut dapat digambarkan pada Gambar 2.2 berikut.



Gambar 2.2 Flowchart Penggunaan Aplikasi oleh Pengguna

2.2 Pengumpulan Dataset

Pengumpulan dataset merupakan tahap pertama dan salah satu tahap terpenting dalam penelitian ini. Kualitas dan keberagaman dataset yang dikumpulkan akan sangat menentukan kemampuan model dalam mengenali kondisi kendaraan secara akurat. Berbeda dengan pendekatan pengumpulan data konvensional yang memerlukan proses pengambilan foto kendaraan nyata secara manual beserta anotasi label yang memakan waktu, penelitian ini memanfaatkan simulator mobil 3D interaktif berbasis

web sebagai sumber data. Pendekatan ini memungkinkan pengumpulan data dilakukan secara otomatis dan terstruktur dengan label ground truth yang diperoleh secara deterministik, sehingga seluruh proses dapat diselesaikan secara efisien tanpa memerlukan anotasi manual.

2.2.1 Sumber Data (Simulator 3D)

Dataset yang digunakan dalam penelitian ini bersumber dari simulator mobil 3D interaktif berbasis web yang dapat diakses pada alamat <http://103.233.100.26:8080/>. Simulator tersebut menampilkan model kendaraan BMW X7 dalam lingkungan 3D yang dapat diinteraksikan melalui browser. Simulator menyediakan lima tombol kontrol yang masing-masing mengatur kondisi satu bagian kendaraan, yaitu *Front Left Door*, *Front Right Door*, *Rear Left Door*, *Rear Right Door*, dan *Hood*. Setiap bagian memiliki dua kondisi, yaitu terbuka (*open*, bernilai 1) dan tertutup (*closed*, bernilai 0).

2.2.2 Otomatisasi dengan Selenium

Proses pengumpulan dataset dilakukan secara otomatis menggunakan skrip Python berbasis Selenium WebDriver yang dikombinasikan dengan Chrome DevTools Protocol (CDP) untuk mensimulasikan event mouse seperti drag rotasi dan scroll zoom secara lebih presisi. Skrip bekerja dengan mengiterasi seluruh 32 konfigurasi kondisi kendaraan, dan untuk setiap konfigurasi dilakukan reset halaman terlebih dahulu untuk memastikan kondisi awal kendaraan kembali ke posisi semua tertutup, kemudian tombol diklik sesuai konfigurasi, zoom diterapkan melalui CDP mouseWheel sebanyak repeat kali, lalu gambar diambil dari 24 sudut rotasi horizontal secara berurutan. Hasil running skrip yang menunjukkan proses iterasi berjalan sesuai konfigurasi dapat dilihat pada Gambar 2.3 berikut.

```
Chrome terbuka.
Membuka URL: http://103.233.100.26:8080
Current URL: http://103.233.100.26:8080/
Total Configurations : 32
Zoom Levels          : 3
Horizontal Steps      : 24
Expected Images       : 2304

-----
[1/32] Config = (0, 0, 0, 0, 0) | bits=00000
-----
Zoom: zoom_out | delta=300 | repeat=6
  Saved: 24/2304 | zoom=zoom_out | angle=23
Zoom: normal | delta=0 | repeat=0
  Saved: 48/2304 | zoom=normal | angle=23
Zoom: zoom_in | delta=-300 | repeat=6
  Saved: 55/2304 | zoom=zoom_in | angle=06
```

Gambar 2.3 Tampilan Running Skrip Pengumpulan Dataset

Pengambilan gambar dilakukan menggunakan JavaScript `canvas.toDataURL()` yang membaca pixel buffer WebGL secara langsung sehingga hasil yang diperoleh murni hanya berisi render 3D kendaraan tanpa elemen UI browser. Gambar kemudian diproses menggunakan metode *letterbox resize* yang mempertahankan aspek rasio asli dengan menambahkan padding background di sisi yang lebih pendek, sehingga perbedaan level zoom tetap terlihat pada gambar akhir. Rotasi kamera dilakukan dengan drag

dari titik background yang aman berdasarkan estimasi warna sudut gambar agar drag tidak secara tidak sengaja menekan area kendaraan yang dapat memicu interaksi yang tidak diinginkan pada simulator. Setiap gambar diberi nama dengan format `cfg_{bits}_{zoom_name}_a{angle_id}.png`, contohnya `cfg_00000_zoom_out_a00.png` dan metadata lengkap disimpan pada file CSV bersama label kondisi kelima bagian kendaraan.

2.2.3 Variasi yang Dikumpulkan (sudut, zoom, konfigurasi)

Untuk memastikan model yang dilatih cukup robust terhadap berbagai kondisi tampilan kendaraan, dataset dirancang mencakup tiga dimensi variasi utama, yaitu variasi konfigurasi kondisi, variasi sudut pandang, dan variasi skala.

1. Variasi Konfigurasi Kondisi

Terdapat 5 bagian kendaraan yang masing-masing memiliki 2 kondisi (terbuka/tertutup), sehingga menghasilkan $2^5 = 32$ kombinasi konfigurasi yang berbeda. Seluruh kombinasi diiterasi secara exhaustive untuk memastikan setiap kombinasi kondisi terwakili dalam dataset, seperti yang ditunjukkan pada Tabel 2.2 berikut.

Tabel 2.2 Variasi Konfigurasi Kondisi pada Pengumpulan Dataset

No.	FL	FR	RL	RR	Hood	Keterangan
1	0	0	0	0	0	Semua tertutup
2	1	0	0	0	0	Hanya FL terbuka
3	0	1	0	0	0	Hanya FR terbuka
4	0	0	1	0	0	Hanya RL terbuka
5	0	0	0	1	0	Hanya RR terbuka
6	0	0	0	0	1	Hanya Hood terbuka
7	1	1	0	0	0	FL + FR terbuka
8	1	0	1	0	0	FL + RL terbuka
9	1	0	0	1	0	FL + RR terbuka
10	1	0	0	0	1	FL + Hood terbuka
11	0	1	1	0	0	FR + RL terbuka
12	0	1	0	1	0	FR + RR terbuka
13	0	1	0	0	1	FR + Hood terbuka
14	0	0	1	1	0	RL + RR terbuka
15	0	0	1	0	1	RL + Hood terbuka
16	0	0	0	1	1	RR + Hood terbuka
17	1	1	1	0	0	FL + FR + RL terbuka
18	1	1	0	1	0	FL + FR + RR terbuka
19	1	1	0	0	1	FL + FR + Hood terbuka
20	1	0	1	1	0	FL + RL + RR terbuka
21	1	0	1	0	1	FL + RL + Hood terbuka
22	1	0	0	1	1	FL + RR + Hood terbuka
23	0	1	1	1	0	FR + RL + RR terbuka

24	0	1	1	0	1	FR + RL + Hood terbuka
25	0	1	0	1	1	FR + RR + Hood terbuka
26	0	0	1	1	1	RL + RR + Hood terbuka
27	1	1	1	1	0	Semua terbuka kecuali Hood
28	1	1	1	0	1	Semua terbuka kecuali RR
29	1	1	0	1	1	Semua terbuka kecuali RL
30	1	0	1	1	1	Semua terbuka kecuali FR
31	0	1	1	1	1	Semua terbuka kecuali FL
32	1	1	1	1	1	Semua terbuka

2. Variasi Sudut Pandang

Untuk setiap konfigurasi, kamera dirotasi secara horizontal sebanyak 24 langkah dengan interval drag sebesar 80 piksel per langkah, sehingga menghasilkan 24 sudut pandang berbeda yang merepresentasikan tampilan kendaraan dari berbagai arah secara merata dalam satu putaran penuh 360°.

3. Variasi Skala (Zoom)

Setiap kombinasi konfigurasi dan sudut pandang diambil pada 3 level zoom yang berbeda. Zoom diterapkan dengan cara melakukan scroll pada canvas sebanyak repeat kali dengan nilai wheel_delta tertentu per scroll. Detail setiap level zoom dapat dilihat pada Tabel 2.3 berikut.

Tabel 2.3 Variasi Level Zoom pada Pengumpulan Dataset

Level	Nilai Scroll	Repeat	Keterangan
1	+300	6	Zoom Out
2	0	0	Zoom Normal (posisi default)
3	-300	6	Zoom In

Total perpindahan scroll efektif untuk zoom out adalah $300 \times 6 = 1800$ unit, sedangkan untuk zoom in adalah $-300 \times 6 = -1800$ unit. Level normal tidak menerapkan scroll sehingga kamera berada pada posisi bawaan simulator.

2.2.4 Auto Crop & Preprocessing Screenshot

Gambar hasil screenshot dari canvas simulator memiliki dimensi yang besar dengan area background gelap yang dominan di sekitar objek kendaraan, sehingga diperlukan proses auto crop sebelum penyimpanan untuk menghasilkan gambar yang bersih dan terfokus pada objek kendaraan. Proses auto crop bekerja dengan cara mengestimasi warna background dari keempat sudut gambar, kemudian menghitung selisih warna setiap pixel terhadap warna background tersebut, di mana pixel dengan selisih warna lebih dari 15 unit dianggap sebagai foreground (bagian kendaraan) dan pixel lainnya dianggap background. Bounding box dari seluruh pixel foreground kemudian diekstrak dengan tambahan padding sebesar 40 pixel di setiap sisi, lalu di-crop dan diletakkan di tengah kanvas persegi dengan warna background yang sama. Selanjutnya gambar diresize ke ukuran 512×512 piksel menggunakan metode interpolasi Lanczos untuk mempertahankan kualitas visual, kemudian disimpan dalam format PNG.

2.2.5 Hasil Pengumpulan Dataset

Proses pengumpulan dataset berhasil diselesaikan dengan total keseluruhan gambar yang terkumpul sesuai dengan yang diharapkan. Total gambar yang dikumpulkan dihitung berdasarkan perkalian seluruh variasi yang diterapkan pada Persamaan (II-1).

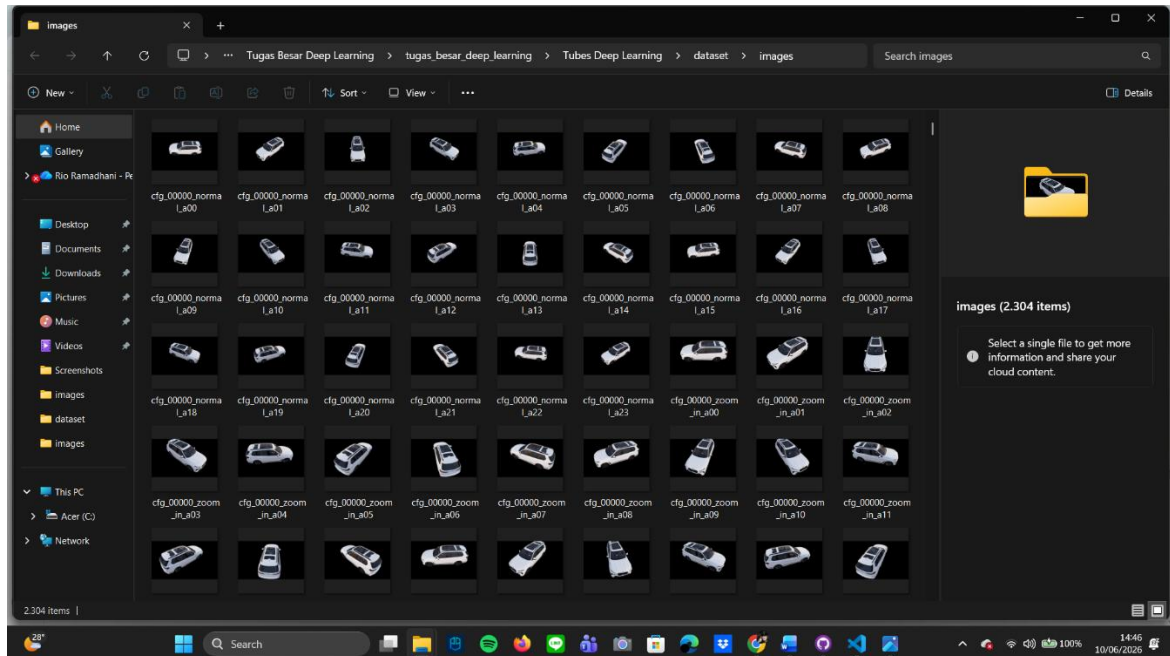
$$N = K \times S \times Z \quad (\text{II-1})$$

di mana:

- N : total gambar
- K : jumlah konfigurasi kondisi = $2^5 = 32$
- S : jumlah sudut rotasi = 24
- Z : jumlah level zoom = 3

$$N = 32 \times 24 \times 3 = 2.304 \text{ gambar}$$

Seluruh gambar tersimpan dalam folder dataset/images/ dengan penamaan yang informatif mengikuti format `cfg_{bits}_{zoom_name}_a{angle_id}.png`. Struktur penamaan ini memudahkan identifikasi setiap gambar berdasarkan konfigurasi kondisi kendaraan, level zoom, dan sudut pengambilan tanpa perlu membuka file CSV. Tampilan folder dataset hasil pengumpulan yang berisi 2.304 item dapat dilihat pada Gambar 2.4 berikut.



Gambar 2.4 Tampilan Folder Dataset Hasil Pengumpulan (2.304 Gambar)

Berdasarkan Gambar 2.4 terlihat bahwa gambar-gambar yang terkumpul menampilkan kendaraan dari berbagai sudut pandang secara konsisten. Pada baris pertama hingga ketiga terlihat gambar dengan level zoom normal (cfg_00000_normal_a00 hingga cfg_00000_normal_a23) yang merepresentasikan 24 sudut rotasi horizontal untuk konfigurasi semua tertutup. Selanjutnya pada baris keempat dan seterusnya terdapat gambar dengan level zoom in (cfg_00000_zoom_in_a00 hingga seterusnya) yang menampilkan kendaraan dengan ukuran lebih besar pada frame. Perbedaan ukuran objek kendaraan antar level zoom terlihat jelas pada gambar, yang mengonfirmasi bahwa metode *letterbox resize* berhasil mempertahankan efek zoom pada gambar akhir. Keseluruhan dataset kemudian disimpan bersama file labels.csv yang memuat informasi label kondisi dan metadata lengkap untuk setiap gambar.

2.3 Preprocessing Dataset

Setelah dataset berhasil dikumpulkan, tahap selanjutnya adalah preprocessing dataset untuk memastikan data yang digunakan dalam proses pelatihan model berkualitas baik, bersih, dan terdistribusi secara proporsional. Tahap preprocessing terdiri dari tiga langkah utama, yaitu validasi data, analisis distribusi label, dan pembagian dataset.

2.3.1 Validasi Data

Validasi data dilakukan untuk memastikan integritas dataset sebelum digunakan dalam proses pelatihan. Proses validasi mencakup tiga aspek utama, yaitu pengecekan nilai yang hilang (*missing values*) pada file CSV, pendeteksian data duplikat berdasarkan nama file gambar, serta verifikasi keberadaan dan integritas setiap file gambar di dalam folder dataset. Hasil running proses validasi dapat dilihat pada Gambar 2.5 berikut.

```
=====
1. LOAD & VALIDASI CSV
=====
Total rows      : 2304
Columns         : ['image', 'front_left', 'front_right', 'rear_left', 'rear_right', 'hood', 'config', 'zoom_id', 'zoom_name', 'zoom_delta', 'angle_id']
Missing values  :
image          : 0
front_left     : 0
front_right    : 0
rear_left      : 0
rear_right     : 0
hood           : 0
config         : 0
zoom_id        : 0
zoom_name      : 0
zoom_delta     : 0
angle_id       : 0
dtype: int64

Duplikat dihapus : 0 rows

=====
2. VALIDASI FILE GAMBAR
=====
Missing files : 0
Corrupt files : 0
Sisa data valid: 2304 rows
```

Gambar 2.5 Hasil Validasi Dataset

Berdasarkan Gambar 2.5 terlihat bahwa seluruh 2.304 data tidak memiliki nilai yang hilang pada kesebelas kolom, yaitu image, front_left, front_right, rear_left, rear_right, hood, config, zoom_id, zoom_name, zoom_delta, dan angle_id. Tidak ditemukan pula data duplikat maupun file gambar yang

hilang atau corrupt, sehingga seluruh 2.304 data dinyatakan valid dan dapat langsung digunakan untuk tahap selanjutnya tanpa diperlukan proses pembersihan data.

2.3.2 Distribusi Label

Analisis distribusi label dilakukan untuk memahami keseimbangan data pada setiap kelas sebelum proses pelatihan dimulai. Distribusi label yang tidak seimbang dapat menyebabkan model cenderung memprediksi kelas mayoritas dan mengabaikan kelas minoritas, sehingga perlu diidentifikasi sejak dini. Hasil analisis distribusi label pada dataset yang dikumpulkan dapat dilihat pada Gambar 2.6 berikut.

```
=====
3. DISTRIBUSI LABEL
=====
front_left      1152
front_right     1152
rear_left       1152
rear_right      1152
hood            1152
dtype: int64
Plot disimpan → dataset/label_distribution.png
```

Gambar 2.6 Hasil Analisis Distribusi Label Dataset

Berdasarkan Gambar 2.6 terlihat bahwa seluruh label memiliki jumlah kemunculan kondisi positif yang identik, yaitu masing-masing sebesar 1.152 dari total 2.304 gambar. Hal ini berarti setiap label memiliki proporsi 50% positif dan 50% negatif, sehingga distribusi label dapat dikatakan seimbang sempurna. Keseimbangan ini merupakan hasil langsung dari desain pengumpulan data yang mengiterasi seluruh 32 kombinasi konfigurasi secara *exhaustive*, sehingga setiap bagian kendaraan memiliki jumlah kemunculan kondisi terbuka dan tertutup yang sama persis. Dengan distribusi yang seimbang, tidak diperlukan teknik penanganan ketidakseimbangan kelas seperti *oversampling*, *undersampling*, maupun penyesuaian `pos_weight` pada fungsi loss selama proses pelatihan.

2.3.3 Verifikasi Ukuran dan Mode Gambar

Selain validasi integritas data, dilakukan pula verifikasi terhadap ukuran dan mode warna seluruh gambar dalam dataset. Verifikasi ini bertujuan untuk memastikan konsistensi format gambar sebelum masuk ke tahap pelatihan, mengingat model deep learning mensyaratkan input dengan dimensi dan format yang seragam. Hasil verifikasi dapat dilihat pada Gambar 2.7 berikut.

```
=====
4. CEK UKURAN & MODE GAMBAR
=====
Mode gambar      : {'RGB'}
Ukuran unik     : {(512, 512)}
✓ Semua gambar ukurannya sama
```

Gambar 2.7 Hasil Verifikasi Ukuran dan Mode Gambar

Berdasarkan Gambar 2.7 terlihat bahwa seluruh gambar dalam dataset memiliki mode warna RGB dan ukuran yang seragam yaitu 512×512 piksel. Keseragaman ini merupakan hasil dari proses *letterbox resize* yang diterapkan pada tahap pengumpulan data, di mana setiap gambar hasil screenshot canvas secara konsisten diubah ukurannya menjadi 512×512 piksel sebelum disimpan. Dengan demikian tidak diperlukan proses konversi mode warna maupun penyeragaman ukuran tambahan sebelum pelatihan, dan seluruh gambar dapat langsung digunakan sebagai input model.

2.3.4 Pembagian Dataset

Setelah validasi dan analisis distribusi selesai dilakukan, dataset dibagi menjadi tiga subset, yaitu data latih (*train*), data validasi (*val*), dan data uji (*test*). Pembagian dilakukan menggunakan metode *stratified split* berdasarkan kombinasi label untuk memastikan setiap subset memiliki distribusi kombinasi konfigurasi yang proporsional. Proporsi pembagian yang digunakan adalah 70% untuk data latih, 20% untuk data validasi, dan 10% untuk data uji. Hasil proses pembagian dan penyalinan dataset ke folder masing-masing subset dapat dilihat pada Gambar 2.8 berikut.

```
=====
5. SPLIT DATASET
=====
Train : 1612 (70.0%)
Val   : 461  (20.0%)
Test  : 231  (10.0%)

=====
6. COPY FILE KE FOLDER SPLIT
=====
✓ Train: 1612 gambar → dataset_split\train
✓ Val: 461 gambar → dataset_split\val
✓ Test: 231 gambar → dataset_split\test
```

Gambar 2.8 Hasil Split Dataset dan Penyalinan ke Folder Subset

Berdasarkan Gambar 2.8 terlihat bahwa proses pembagian dataset menghasilkan 1.612 gambar untuk data latih (70%), 461 gambar untuk data validasi (20%), dan 231 gambar untuk data uji (10%). Seluruh gambar dari masing-masing subset berhasil disalin ke folder terpisah, yaitu `dataset_split\train`, `dataset_split\val`, dan `dataset_split\test`, masing-masing beserta file `labels.csv` yang sesuai. Distribusi label pada masing-masing subset setelah pembagian dapat dilihat pada Gambar 2.9 berikut.

```

=====
7. SUMMARY
=====

[Train]
front_left      806
front_right     808
rear_left       805
rear_right      802
hood            805

[Val]
front_left      231
front_right     230
rear_left       233
rear_right      232
hood            229

[Test]
front_left      115
front_right     114
rear_left       114
rear_right      118
hood            118

✓ Preprocessing selesai!
Output folder : dataset_split/
├── train/images/ (1612 files)
├── train/labels.csv
├── val/images/ (461 files)
├── val/labels.csv
├── test/images/ (231 files)
└── test/labels.csv

```

Gambar 2.9 Distribusi Label per Subset

Berdasarkan Gambar 2.9 terlihat bahwa distribusi label pada setiap subset tetap terjaga dengan baik setelah proses pembagian, di mana jumlah label positif pada masing-masing subset mendekati setengah dari total gambar pada subset tersebut. Hal ini mengonfirmasi bahwa metode *stratified split* berhasil mempertahankan keseimbangan distribusi label pada ketiga subset sehingga proses evaluasi model dapat dilakukan secara objektif dan representatif.

2.4 Augmentasi Data

Augmentasi data merupakan teknik untuk memperbesar variasi data latih secara artifisial dengan menerapkan transformasi tertentu pada gambar asli, sehingga model yang dilatih menjadi lebih robust terhadap variasi visual yang mungkin ditemui saat inferensi. Augmentasi diterapkan hanya pada data latih dan tidak diterapkan pada data validasi maupun data uji, agar proses evaluasi tetap mencerminkan kondisi data yang sesungguhnya.

2.4.1 Pemilihan Teknik Augmentasi

Pemilihan teknik augmentasi pada penelitian ini dilakukan secara selektif dengan mempertimbangkan karakteristik dataset yang bersumber dari simulator 3D. Tidak semua teknik augmentasi aman diterapkan pada dataset ini karena beberapa teknik dapat merusak informasi visual yang justru menjadi target prediksi, seperti kondisi pintu dan kap mesin. Evaluasi terhadap setiap teknik augmentasi yang dipertimbangkan dapat dilihat pada Tabel 2.4 berikut.

Tabel 2.4 Evaluasi Teknik Augmentasi

Teknik Augmentasi	Status	Alasan
Resize	Aman	Menyamakan ukuran input gambar sebelum masuk ke model
ShiftScaleRotate ringan	Aman	Membantu model lebih robust terhadap pergeseran posisi, sedikit perubahan skala, dan rotasi kecil
RandomBrightnessContrast ringan	Aman	Membantu model mengenali objek pada variasi pencahayaan dan kontras
Normalize	Aman	Menyesuaikan distribusi piksel dengan standar input model pretrained ImageNet
ToTensorV2	Aman	Mengubah gambar menjadi tensor agar dapat diproses oleh PyTorch
Perspective ringan	Cukup aman	Dapat mensimulasikan sedikit perubahan sudut kamera, tetapi tidak boleh terlalu besar agar bentuk mobil tidak terdistorsi
GaussNoise ringan	Cukup aman	Bisa menambah robustness terhadap noise, tetapi harus sangat ringan agar detail pintu dan hood tidak rusak
HueSaturationValue ringan	Cukup aman	Bisa digunakan sedikit untuk variasi warna, tetapi tidak terlalu penting karena dataset berasal dari objek 3D yang relatif konsisten
HorizontalFlip	Tidak aman	Dapat menukar posisi kiri dan kanan mobil sehingga label front_left, front_right, rear_left, dan rear_right menjadi salah jika label tidak ikut ditukar
CoarseDropout	Tidak aman	Dapat menutup bagian penting seperti pintu atau hood yang menjadi target prediksi
MotionBlur berat	Tidak aman	Dapat mengaburkan garis pintu, celah hood, dan detail kecil yang penting untuk klasifikasi
GaussNoise berat	Tidak aman	Noise berlebih dapat merusak detail visual pada pintu dan kap mesin
Perspective berat	Tidak aman	Distorsi besar dapat membuat bentuk mobil tidak natural dan membingungkan model
CLAHE	Kurang disarankan	Dapat mengubah kontras secara tidak natural pada gambar mobil 3D

Berdasarkan evaluasi pada Tabel 2.4, teknik augmentasi yang dipilih untuk diterapkan adalah teknik-teknik dengan status aman, yaitu *ShiftScaleRotate* ringan dan *RandomBrightnessContrast* ringan, di samping *Resize*, *Normalize*, dan *ToTensorV2* yang bersifat wajib. Teknik dengan status tidak aman seperti *HorizontalFlip* dan *CoarseDropout* sengaja tidak digunakan karena berpotensi merusak konsistensi antara gambar dan label kondisi kendaraan.

2.4.2 Pemilihan Teknik Augmentasi

Augmentasi data diimplementasikan menggunakan library Albumentations dengan parameter yang telah disesuaikan agar transformasi yang diterapkan cukup memberikan variasi tanpa merusak

informasi visual yang relevan. Implementasi augmentasi pada data latih dapat dilihat pada Gambar 2.10 berikut.

```
# =====  
# AUGMENTASI TRAIN – AMAN & TERKONTROL  
# =====  
  
train_transform = A.Compose([  
    A.Resize(IMG_SIZE, IMG_SIZE),  
  
    A.ShiftScaleRotate(  
        shift_limit=0.03,  
        scale_limit=0.05,  
        rotate_limit=8,  
        border_mode=cv2.BORDER_CONSTANT,  
        value=(32, 32, 32),  
        p=0.35  
    ),  
  
    A.RandomBrightnessContrast(  
        brightness_limit=0.10,  
        contrast_limit=0.10,  
        p=0.30  
    ),  
  
    A.Normalize(  
        mean=[0.485, 0.456, 0.406],  
        std=[0.229, 0.224, 0.225]  
    ),  
  
    ToTensorV2()  
)
```

Gambar 2.10 Implementasi Augmentasi Data Latih

Berdasarkan Gambar 2.10, pipeline augmentasi data latih terdiri dari lima tahap. Pertama, *Resize* untuk menyeragamkan ukuran input menjadi 224×224 piksel sesuai kebutuhan EfficientNet-B0. Kedua, *ShiftScaleRotate* dengan parameter *shift_limit*=0.03, *scale_limit*=0.05, dan *rotate_limit*=8 dengan probabilitas 0.35, yang memberikan variasi pergeseran, perubahan skala, dan rotasi kecil secara ringan. Ketiga, *RandomBrightnessContrast* dengan *brightness_limit*=0.10 dan *contrast_limit*=0.10 dengan probabilitas 0.30 untuk mensimulasikan variasi pencahayaan. Keempat, *Normalize* dengan nilai mean [0.485, 0.456, 0.406] dan std [0.229, 0.224, 0.225] yang merupakan standar normalisasi ImageNet sesuai dengan bobot pretrained yang digunakan. Kelima, *ToTensorV2* untuk mengubah gambar menjadi tensor PyTorch. Sementara untuk data validasi dan data uji, pipeline hanya terdiri dari *Resize*, *Normalize*, dan *ToTensorV2* tanpa augmentasi apapun.

2.5 Arsitektur Model

Arsitektur model yang dibangun dalam penelitian ini dirancang untuk menyelesaikan tugas *multi-label classification* terhadap lima kondisi bagian kendaraan secara simultan. Model terdiri dari dua komponen utama, yaitu *backbone* EfficientNet-B0 yang bertugas mengekstrak fitur visual dari gambar input, dan *custom classification head* yang bertugas memetakan fitur tersebut menjadi prediksi probabilitas untuk masing-masing label. Pendekatan *transfer learning* diterapkan dengan memanfaatkan bobot pretrained ImageNet pada backbone, sehingga model tidak perlu belajar dari awal dan proses konvergensi dapat

dicapai lebih cepat meskipun ukuran dataset relatif terbatas. Selain arsitektur model, pada sub-bab ini juga dijelaskan konfigurasi fungsi loss, optimizer, dan scheduler yang digunakan selama proses pelatihan.

2.5.1 Backbone EfficientNet-B0

Model yang digunakan dalam penelitian ini adalah EfficientNet-B0 dengan pendekatan *transfer learning* menggunakan bobot pretrained dari ImageNet. EfficientNet-B0 dipilih sebagai backbone karena memiliki keseimbangan yang baik antara jumlah parameter dan akurasi dengan total 4.369.793 parameter, model ini tergolong ringan namun mampu mengekstrak fitur visual yang kaya dan representatif. EfficientNet-B0 menggunakan prinsip *compound scaling* yang menskalakan kedalaman, lebar, dan resolusi jaringan secara serentak, sehingga efisiensi komputasi tetap terjaga. Arsitektur backbone terdiri dari lapisan konvolusi awal (*conv_stem*), diikuti oleh serangkaian blok *MBConv* dengan mekanisme *Squeeze-and-Excitation* (SE) yang memungkinkan model memperhatikan channel fitur yang paling relevan secara adaptif, dan diakhiri dengan lapisan *global average pooling* yang menghasilkan vektor fitur berdimensi 1.280. Pada penelitian ini, *classification head* bawaan EfficientNet-B0 dihapus (*num_classes=0*) dan digantikan dengan *custom head* yang dirancang sesuai kebutuhan tugas *multi-label classification*.

2.5.2 Custom Classification Head

Vektor fitur berdimensi 1.280 yang dihasilkan backbone kemudian diteruskan ke *custom classification head* yang terdiri dari tiga lapisan *fully connected* dengan *Batch Normalization*, aktivasi ReLU, dan *Dropout* sebagai regularisasi. Arsitektur *custom head* dapat dilihat pada Gambar 2.11 berikut.

```
self.head = nn.Sequential(  
    nn.Linear(in_features, 256),  
    nn.BatchNorm1d(256),  
    nn.ReLU(),  
    nn.Dropout(0.4),  
    nn.Linear(256, 128),  
    nn.BatchNorm1d(128),  
    nn.ReLU(),  
    nn.Dropout(0.3),  
    nn.Linear(128, num_classes)  
)
```

Gambar 2.11 Implementasi Arsitektur Model CarPartClassifier

Berdasarkan Gambar 2.11, *custom classification head* terdiri dari tiga blok lapisan. Blok pertama terdiri dari *Linear*(1280 $\rightarrow$ 256), *BatchNorm1d*(256), ReLU, dan *Dropout*(0.4). Blok kedua terdiri dari *Linear*(256 $\rightarrow$ 128), *BatchNorm1d*(128), ReLU, dan *Dropout*(0.3). Blok ketiga merupakan lapisan output *Linear*(128 $\rightarrow$ 5) yang menghasilkan 5 nilai logit sesuai dengan jumlah label yang diprediksi. Nilai *Dropout* yang lebih besar pada blok pertama (0.4) dibandingkan blok kedua (0.3) bertujuan untuk memberikan regularisasi yang lebih kuat pada lapisan dengan dimensi yang lebih besar.

2.5.3 Fungsi Loss, Optimizer, dan Scheduler

Konfigurasi fungsi loss, optimizer, dan scheduler yang digunakan dalam proses pelatihan dapat dilihat pada Gambar 2.12 berikut.

```
# Label seimbang (50:50), pos_weight = 1.0 semua
pos_weight = torch.ones(NUM_CLASSES).to(DEVICE)

criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)

optimizer = optim.AdamW(
    model.parameters(),
    lr=LR,
    weight_decay=1e-4
)

scheduler = optim.lr_scheduler.CosineAnnealingLR(
    optimizer,
    T_max=NUM_EPOCHS,
    eta_min=1e-6
)
```

Gambar 2.12 Implementasi Loss, Optimizer, dan Scheduler

1. Fungsi Loss

Fungsi loss yang digunakan adalah BCEWithLogitsLoss (*Binary Cross Entropy with Logits Loss*) yang menggabungkan fungsi sigmoid dan *binary cross entropy* dalam satu operasi yang lebih stabil secara numeris. Fungsi loss ini dipilih karena sesuai untuk tugas *multi-label classification* di mana setiap label diperlakukan sebagai klasifikasi biner yang independen. Karena distribusi label pada dataset seimbang sempurna (50:50), nilai `pos_weight` ditetapkan sebesar 1.0 untuk seluruh kelas sehingga tidak diperlukan pembobotan khusus.

2. Optimizer

Optimizer yang digunakan adalah AdamW dengan learning rate sebesar 1×10^{-4} dan weight decay sebesar 1×10^{-4} . AdamW dipilih karena merupakan varian Adam yang memisahkan weight decay dari proses pembaruan gradien, sehingga regularisasi L2 bekerja lebih efektif dibandingkan Adam standar.

3. Learning Rate Scheduler

Scheduler yang digunakan adalah CosineAnnealingLR dengan '`T_max`' sebesar jumlah epoch maksimal dan '`eta_min`' sebesar 1×10^{-6} . Scheduler ini menurunkan learning rate secara bertahap mengikuti fungsi cosinus dari nilai awal menuju nilai minimum, sehingga model dapat melakukan eksplorasi yang lebih luas pada awal pelatihan dan konvergensi yang lebih halus pada akhir pelatihan.

2.6 Training Model

Proses pelatihan model dilakukan menggunakan pipeline yang terdiri dari fungsi `train_one_epoch` untuk pelatihan satu epoch dan fungsi `evaluate` untuk evaluasi pada data validasi. Pada setiap epoch, model dilatih dengan seluruh data latih secara batch, kemudian dievaluasi pada data validasi untuk memantau

performa generalisasi model. Metrik utama yang digunakan untuk memantau performa selama pelatihan adalah *Macro F1-Score* dengan threshold default 0.5. Konfigurasi lengkap hyperparameter, optimizer, scheduler, dan early stopping yang digunakan dapat dilihat pada Tabel 2.5 berikut.

Tabel 2.5 Konfigurasi Pelatihan Model

Komponen	Parameter	Nilai
Hyperparameter	Ukuran input	224×224 piksel
	Batch size	32
	Jumlah epoch maksimal	30
	Threshold default	0.5
Optimizer	Jenis	AdamW
	Learning rate	1×10^{-4}
	Weight decay	1×10^{-4}
Scheduler	Jenis	CosineAnnealingLR
	T_max	30 (= jumlah epoch)
	eta_min	1×10^{-6}
Early Stopping	Patience	5 epoch
	Monitor	Val F1-Score
	Mode	Maksimum
Checkpoint	Kriteria simpan	Val F1 tertinggi
	Format	.pth

Implementasi training loop menggunakan dua fungsi utama. Pada fungsi `train_one_epoch`, model diset ke mode *train* sehingga lapisan *Dropout* dan *BatchNorm* bekerja sesuai perilaku pelatihan, kemudian dilakukan *forward pass*, perhitungan loss, *backward pass*, dan pembaruan bobot melalui optimizer. Pada fungsi `evaluate`, model diset ke mode *eval* dan seluruh komputasi dilakukan dalam konteks `torch.no_grad()` untuk menonaktifkan perhitungan gradien sehingga proses evaluasi lebih efisien. Model terbaik disimpan secara otomatis ke file checkpoint setiap kali terjadi peningkatan *Val F1* dibandingkan epoch sebelumnya, dan mekanisme *early stopping* menghentikan pelatihan apabila *Val F1* tidak mengalami peningkatan selama 5 epoch berturut-turut. Hasil running proses pelatihan dapat dilihat pada Gambar 2.13 berikut.

```

... Epoch [28/30] Train Loss: 0.1671 Train F1: 0.9897 | Val Loss: 0.1369 Val F1: 0.9996
✓ Best model saved (Val F1: 0.9996)

... Training:  0%|          | 0/51 [00:00<?, ?it/s]
... Evaluating: 0%|          | 0/15 [00:00<?, ?it/s]

... Epoch [29/30] Train Loss: 0.1689 Train F1: 0.9872 | Val Loss: 0.1358 Val F1: 0.9991
... Training:  0%|          | 0/51 [00:00<?, ?it/s]
... Evaluating: 0%|          | 0/15 [00:00<?, ?it/s]

... Epoch [30/30] Train Loss: 0.1673 Train F1: 0.9847 | Val Loss: 0.1374 Val F1: 0.9991

```

Gambar 2.13 Hasil Running Proses Pelatihan Model

Berdasarkan Gambar 2.13 terlihat bahwa proses pelatihan berhasil diselesaikan hingga epoch ke-30 tanpa terpicu *early stopping*. Pada epoch terakhir, model mencapai *Train Loss* sebesar 0.1673 dengan *Train F1* sebesar 0.9847, serta *Val Loss* sebesar 0.1374 dengan *Val F1* sebesar 0.9991. Model terbaik tersimpan pada epoch ke-28 dengan *Val F1* tertinggi sebesar 0.9996. Nilai *Val F1* yang lebih tinggi dibandingkan *Train F1* mengindikasikan bahwa model tidak mengalami *overfitting* dan memiliki kemampuan generalisasi yang baik terhadap data yang belum pernah dilihat sebelumnya.

2.7 Evaluasi Model

Evaluasi model dilakukan untuk mengukur kemampuan model dalam melakukan prediksi pada data yang belum pernah dilihat sebelumnya. Pada penelitian ini, evaluasi dilakukan menggunakan *test set* yang terdiri dari 231 gambar dengan total 1.155 label prediksi (231×5 label). Proses evaluasi menggunakan nilai *threshold* default sebesar 0,5 untuk menentukan kelas positif pada setiap label. Evaluasi mencakup empat aspek utama, yaitu metrik keseluruhan, performa per label, analisis *confusion matrix*, serta visualisasi prediksi sampel secara kualitatif.

2.7.1 Metrik Keseluruhan (Macro F1, Micro F1, Precision, Recall, Hamming Loss)

Evaluasi model dilakukan pada data uji (*test set*) yang terdiri dari 231 gambar yang tidak pernah digunakan selama proses pelatihan maupun validasi. Pada tahap evaluasi, prediksi kelas ditentukan menggunakan nilai *threshold* default sebesar 0,5 untuk setiap label. Hasil evaluasi keseluruhan pada *test set* dapat dilihat pada Gambar 2.14 berikut.

```
=====
EVALUASI TEST SET
=====
Hamming Loss   : 0.0017
Macro F1       : 0.9983
Micro F1       : 0.9983
Macro Precision: 0.9967
Macro Recall   : 1.0000
```

Gambar 2.14 Hasil Evaluasi Keseluruhan pada Test Set

Berdasarkan Gambar 2.14, model mencapai performa yang sangat baik pada seluruh metrik evaluasi. Hamming Loss sebesar 0,0017 menunjukkan bahwa hanya sekitar 0,17% prediksi label yang salah dari keseluruhan prediksi, sehingga hampir seluruh label berhasil diprediksi dengan benar. Nilai Macro F1 dan Micro F1 yang masing-masing sebesar 0,9983 menunjukkan keseimbangan yang sangat baik antara *precision* dan *recall*, baik pada setiap label maupun secara keseluruhan. Selain itu, Macro Recall sebesar 1,0000 menunjukkan bahwa model berhasil mendeteksi seluruh kondisi positif tanpa ada yang terlewat, sedangkan Macro Precision sebesar 0,9967 mengindikasikan bahwa hanya terdapat sedikit prediksi positif yang salah. Hasil ini menunjukkan bahwa model memiliki kemampuan klasifikasi yang sangat tinggi dalam mengenali seluruh kondisi kendaraan pada dataset pengujian.

2.7.2 Performa per Label (Classification Report)

Evaluasi performa model secara per label dilakukan menggunakan *classification report* yang menampilkan nilai *precision*, *recall*, dan *F1-score* untuk masing-masing kelas. Hasil *classification report* pada test set dapat dilihat pada Gambar 2.15 berikut.

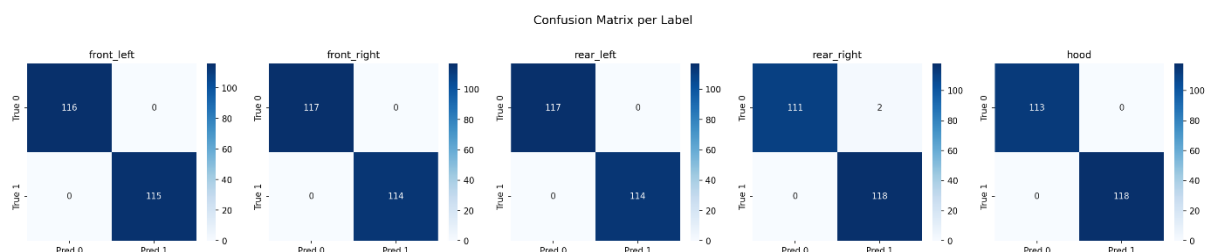
Per-class Report:				
	precision	recall	f1-score	support
front_left	1.00	1.00	1.00	115
front_right	1.00	1.00	1.00	114
rear_left	1.00	1.00	1.00	114
rear_right	0.98	1.00	0.99	118
hood	1.00	1.00	1.00	118
micro avg	1.00	1.00	1.00	579
macro avg	1.00	1.00	1.00	579
weighted avg	1.00	1.00	1.00	579
samples avg	0.97	0.97	0.97	579

Gambar 2.15 Hasil Classification Report Per-class

Berdasarkan Gambar 2.15, hampir seluruh label mencapai nilai *precision*, *recall*, dan *F1-score* sempurna sebesar 1.00, kecuali label *rear_right* yang memiliki *precision* sedikit lebih rendah yaitu 0.98 dengan *F1-score* 0.99. Hal ini mengindikasikan terdapat sedikit prediksi *false positive* pada label *rear_right*, yaitu model sesekali memprediksi pintu belakang kanan dalam kondisi terbuka padahal kondisi sebenarnya tertutup. Secara keseluruhan seluruh label mencapai nilai *recall* sempurna 1.00 yang berarti model tidak melewatkan satu pun kondisi positif pada seluruh kelas.

2.7.3 Confusion Matrix per Label

Analisis lebih mendalam terhadap performa model dilakukan melalui *confusion matrix* per label yang menampilkan distribusi prediksi benar dan salah untuk setiap kelas. Hasil *confusion matrix* per label dapat dilihat pada Gambar 2.16 berikut.



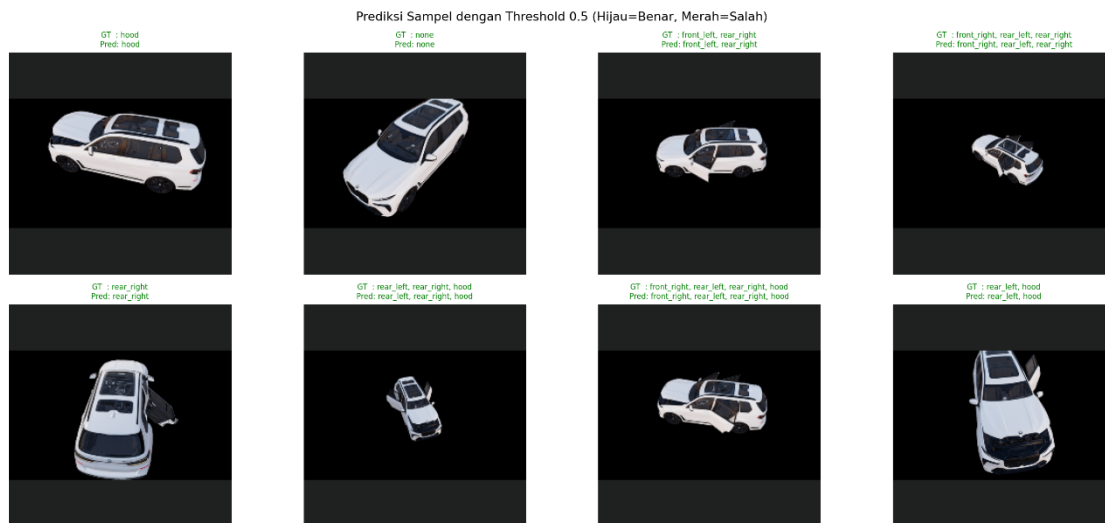
Gambar 2.16 Hasil Confusion Matrix per Label

Berdasarkan Gambar 4.X terlihat bahwa hampir seluruh prediksi terkonsentrasi pada diagonal utama yang merepresentasikan prediksi benar. Secara rinci, label 'front_left' menunjukkan 116 prediksi *True Negative* dan 115 prediksi *True Positive* tanpa kesalahan sama sekali. Label 'front_right' menunjukkan

117 *True Negative* dan 114 *True Positive* tanpa kesalahan. Label 'rear_left' menunjukkan 117 *True Negative* dan 114 *True Positive* tanpa kesalahan. Label 'rear_right' menunjukkan 111 *True Negative*, 118 *True Positive*, namun terdapat 2 prediksi *False Positive* yang menjadi satu-satunya kesalahan pada seluruh test set. Label 'hood' menunjukkan 113 *True Negative* dan 118 *True Positive* tanpa kesalahan. Total kesalahan prediksi pada seluruh test set hanya berjumlah 2 dari keseluruhan $231 \times 5 = 1.155$ prediksi label, yang mengonfirmasi nilai *Hamming Loss* sebesar 0.0017.

2.7.4 Prediksi Sample

Untuk memberikan gambaran kualitatif terhadap performa model, dilakukan visualisasi prediksi pada beberapa sampel gambar dari test set. Hasil visualisasi prediksi sampel dapat dilihat pada Gambar 2.17 berikut.



Gambar 2.17 Hasil Visualisasi Prediksi Sampel (Hijau = Benar, Merah = Salah)

Berdasarkan Gambar 2.17 terlihat bahwa seluruh 8 sampel yang ditampilkan diprediksi dengan benar oleh model, ditandai dengan warna hijau pada seluruh judul gambar. Model berhasil memprediksi berbagai kombinasi kondisi kendaraan secara akurat, mulai dari kondisi hanya satu bagian terbuka seperti hood saja, kondisi semua tertutup (none), hingga kombinasi beberapa bagian terbuka sekaligus seperti front_right, rear_left, rear_right, hood. Model juga menunjukkan kemampuan yang baik dalam mengenali kondisi kendaraan dari berbagai sudut pandang dan level zoom yang berbeda, termasuk sudut atas (*top view*) maupun sudut samping (*side view*). Hal ini mengonfirmasi bahwa augmentasi data yang diterapkan selama pelatihan berhasil meningkatkan robustness model terhadap variasi visual pada dataset.

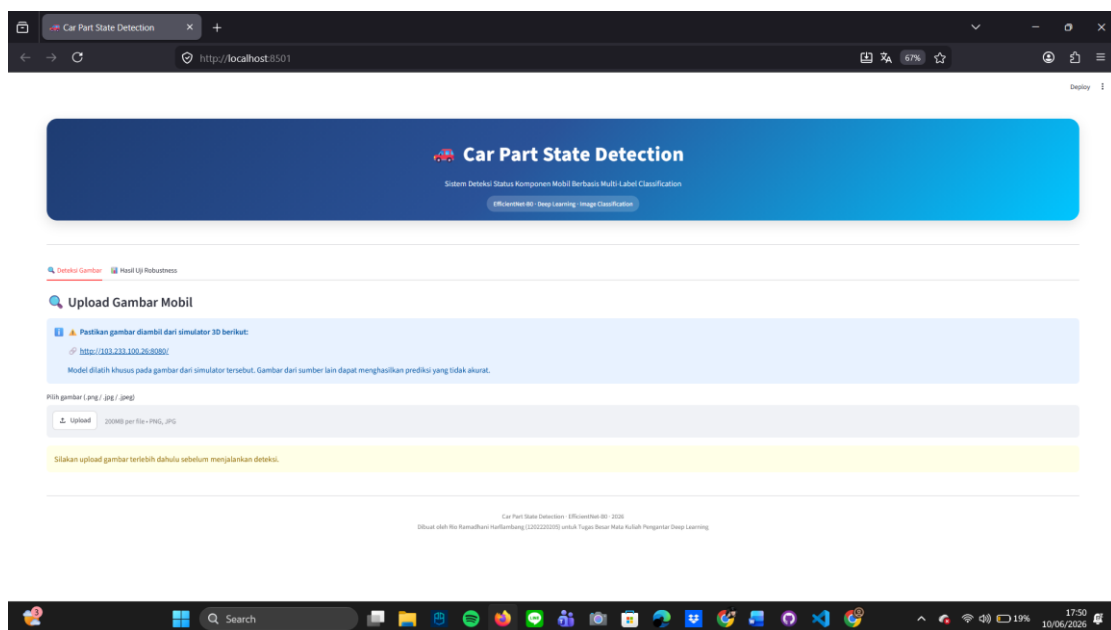
2.8 Deployment

Setelah model EfficientNet-B0 berhasil dilatih dan dievaluasi, tahap selanjutnya adalah melakukan *deployment* agar model dapat digunakan secara langsung oleh pengguna melalui aplikasi berbasis web.

Pada penelitian ini, *deployment* dilakukan menggunakan framework Streamlit yang memungkinkan integrasi model deep learning dengan antarmuka pengguna yang interaktif dan mudah digunakan.

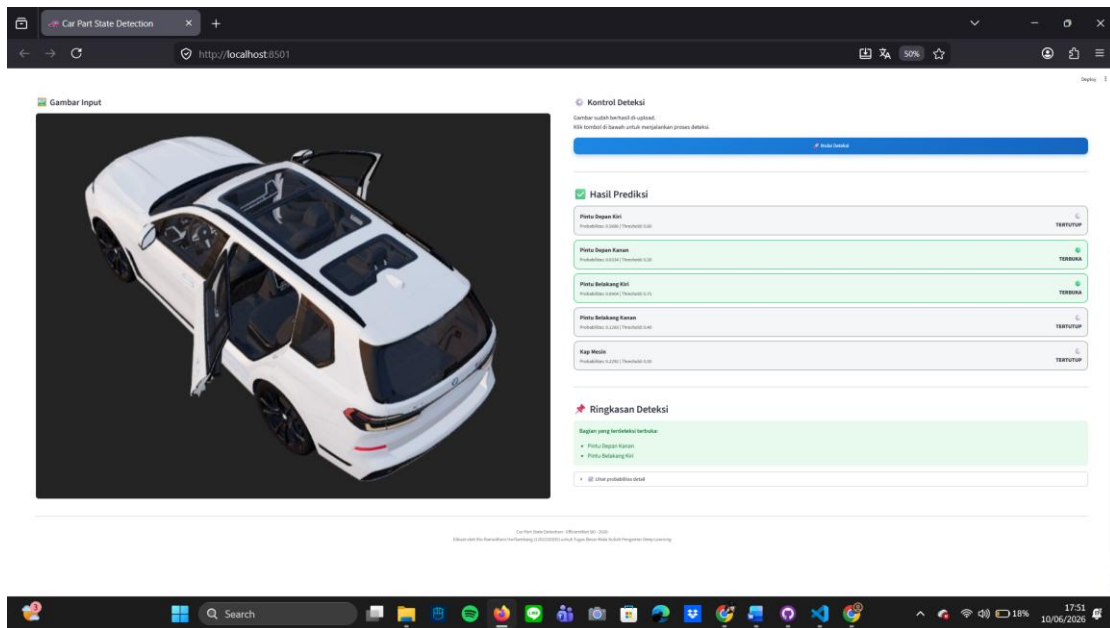
Model terbaik hasil pelatihan disimpan dalam file *checkpoint* dan dimuat kembali pada saat aplikasi dijalankan. Pengguna dapat mengunggah gambar mobil melalui antarmuka aplikasi, kemudian sistem akan melakukan *preprocessing* menggunakan transformasi yang sama dengan proses pengujian model. Selanjutnya, gambar diproses oleh model EfficientNet-B0 untuk menghasilkan probabilitas pada lima label komponen kendaraan, yaitu pintu depan kiri, pintu depan kanan, pintu belakang kiri, pintu belakang kanan, dan kap mesin. Probabilitas tersebut kemudian dikonversi menjadi status terbuka atau tertutup menggunakan nilai *threshold* yang telah ditentukan.

Tampilan halaman utama aplikasi ditunjukkan pada Gambar 2.18. Halaman ini berisi informasi mengenai sistem deteksi, petunjuk penggunaan aplikasi, serta fitur unggah gambar yang digunakan sebagai masukan model.



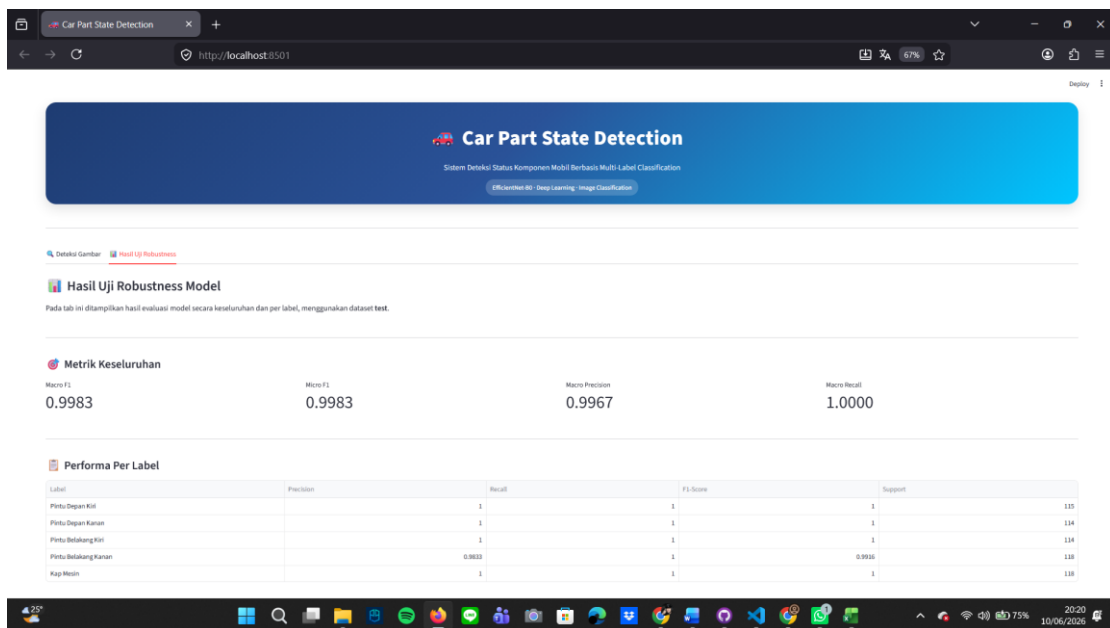
Gambar 2.18 Halaman Utama Aplikasi Deployment

Setelah gambar berhasil diunggah, pengguna dapat menjalankan proses deteksi dengan menekan tombol Mulai Deteksi. Sistem kemudian melakukan inferensi dan menampilkan hasil prediksi untuk setiap komponen kendaraan beserta nilai probabilitas yang dihasilkan model. Selain itu, sistem juga menampilkan ringkasan komponen yang terdeteksi dalam kondisi terbuka sehingga hasil prediksi dapat dipahami dengan lebih mudah oleh pengguna. Contoh hasil prediksi ditunjukkan pada Gambar 2.19.



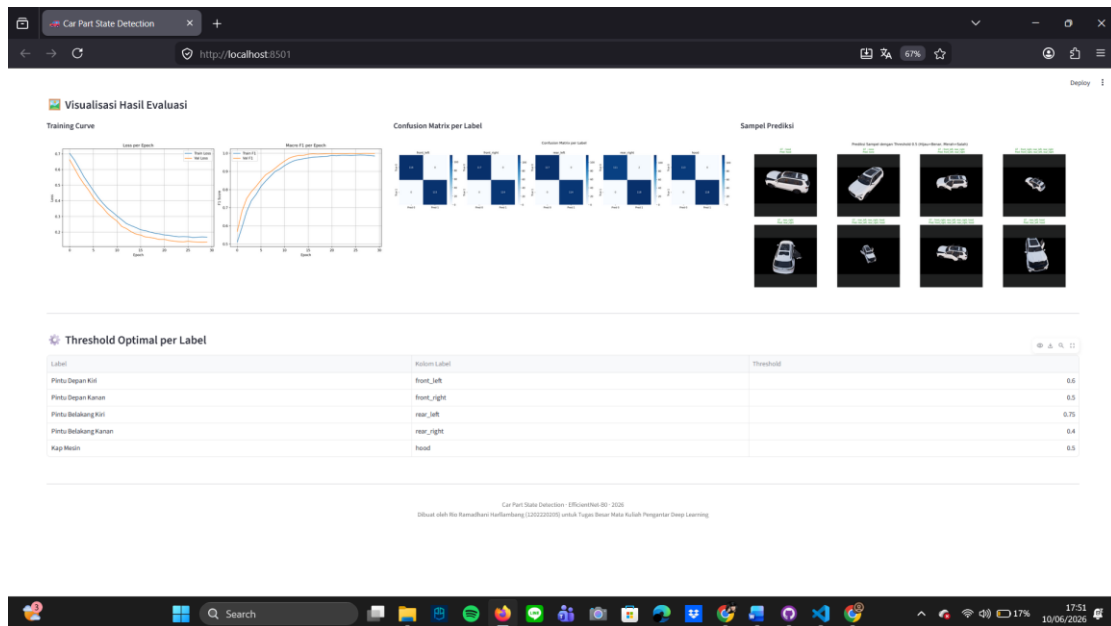
Gambar 2.19 Hasil Prediksi pada Aplikasi Deployment

Selain menyediakan fitur deteksi gambar, aplikasi juga dilengkapi dengan halaman Hasil Uji Robustness Model yang menampilkan ringkasan performa model pada data uji. Informasi yang ditampilkan meliputi nilai Macro F1-Score, Micro F1-Score, Macro Precision, dan Macro Recall, serta tabel performa untuk setiap label. Tampilan halaman hasil evaluasi ditunjukkan pada Gambar 2.20.



Gambar 2.20 Halaman Hasil Uji Robustness Model

Sebagai pendukung analisis performa, aplikasi juga menampilkan visualisasi hasil pelatihan dan pengujian model, seperti *training curve*, *confusion matrix* per label, dan contoh hasil prediksi pada data uji. Visualisasi tersebut membantu pengguna memahami performa model secara lebih komprehensif. Tampilan visualisasi hasil evaluasi ditunjukkan pada Gambar 2.21.



Gambar 2.21 Visualisasi Hasil Evaluasi Model

Berdasarkan hasil implementasi, model EfficientNet-B0 berhasil diintegrasikan ke dalam aplikasi web berbasis Streamlit dan mampu melakukan deteksi status komponen kendaraan secara otomatis. Sistem dapat menerima masukan berupa gambar kendaraan, melakukan proses inferensi, serta menampilkan hasil prediksi secara real-time dengan antarmuka yang mudah digunakan. Dengan demikian, proses *deployment* yang dilakukan telah berhasil menerapkan model deep learning ke dalam aplikasi yang siap digunakan untuk mendukung deteksi kondisi komponen kendaraan.

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan hasil penelitian yang telah dilakukan, dapat disimpulkan bahwa model *multi-label image classification* menggunakan arsitektur EfficientNet-B0 berhasil dikembangkan untuk mendeteksi status lima komponen kendaraan, yaitu pintu depan kiri, pintu depan kanan, pintu belakang kiri, pintu belakang kanan, dan kap mesin dalam kondisi terbuka atau tertutup. Model menunjukkan performa yang sangat baik pada data uji dengan nilai Macro F1-Score sebesar 0,9983, Micro F1-Score sebesar 0,9983, Macro Precision sebesar 0,9967, Macro Recall sebesar 1,0000, dan Hamming Loss sebesar 0,0017. Selain itu, model berhasil diimplementasikan ke dalam aplikasi berbasis web menggunakan Streamlit yang mampu melakukan deteksi secara otomatis dan menampilkan hasil prediksi secara real-time. Hasil penelitian ini menunjukkan bahwa metode yang diusulkan efektif untuk mendukung proses deteksi status komponen kendaraan berbasis citra digital.

3.2 Saran

Berdasarkan hasil penelitian yang telah dilakukan, beberapa saran untuk pengembangan penelitian selanjutnya adalah sebagai berikut:

1. Menambahkan variasi dataset dengan kondisi pencahayaan, sudut pengambilan gambar, dan jenis kendaraan yang lebih beragam untuk meningkatkan kemampuan generalisasi model.
2. Melakukan pengujian menggunakan gambar kendaraan dari lingkungan nyata (*real-world images*) untuk mengetahui performa model di luar data simulator.
3. Mencoba arsitektur deep learning lain, seperti EfficientNetV2, ConvNeXt, atau Vision Transformer (ViT), sebagai pembanding terhadap performa EfficientNet-B0.
4. Menambahkan deteksi pada komponen kendaraan lainnya, seperti bagasi, jendela, atau lampu kendaraan agar sistem menjadi lebih komprehensif.
5. Mengembangkan aplikasi ke platform mobile atau mengintegrasikannya dengan sistem pemantauan kendaraan secara real-time.

LAMPIRAN

Link Repository Github :

https://github.com/Tayo1103/tugas_besar_deep_learning.git

Link Google Drive :

https://drive.google.com/drive/folders/1Q-uAhIQWa6BgKrwIjpZIyqWbKya7c2wE?usp=drive_link